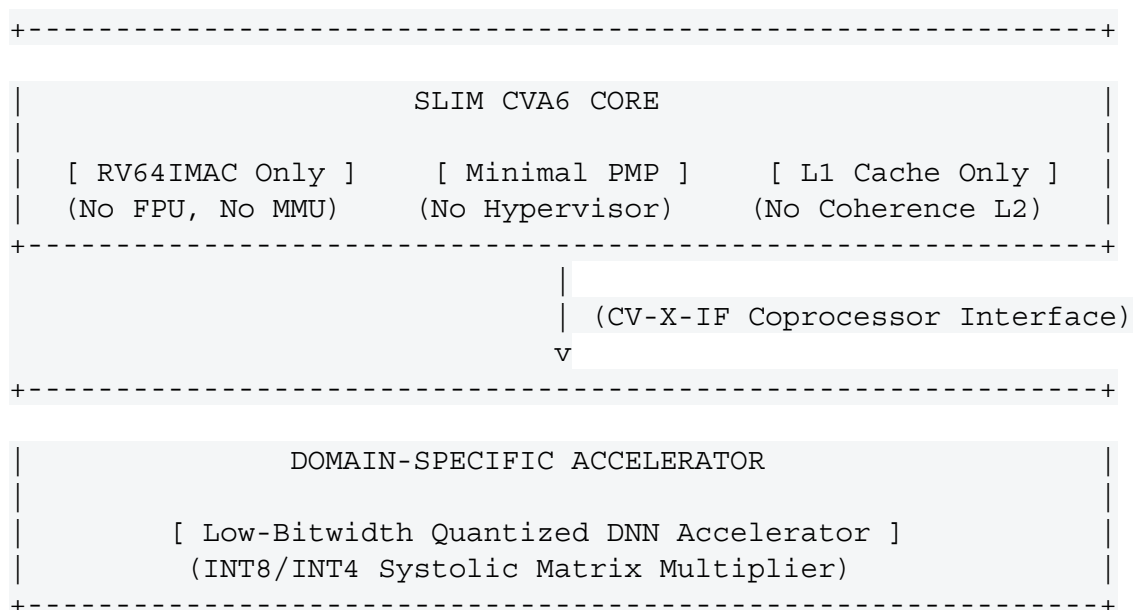


Project Title: "The Slim-CVA6 Edge-AI SoC"

Core Objective: Strip the CVA6 core down to its bare essential control plane logic by removing virtualization, heavy floating-point, and translation circuits, and replace them with a dedicated **Gemmini-like Matrix Multiplication Unit** using the **CV-X-IF** coprocessor interface. [4, 9, 10]



Phase 1: What Circuits to Remove (The Slimming Process)

To convert CVA6 into a specialized "Edge-AI" microcontroller controller, strip out logic that handles general-purpose desktop/server workloads:

- **Remove the Floating-Point Unit (FPU - 'F' & 'D' Extensions):** High-precision 32/64-bit floating-point units take up massive silicon real estate. For an edge accelerator, complex mathematical operations are pushed to the custom accelerator. [11, 12]
- **Remove the Memory Management Unit (MMU) & Page Table Walker (PTW):** Virtual memory address translation adds massive pipeline timing uncertainties and circuit area. Replace it with simple, static Physical Memory Protection (PMP) blocks. [13, 14, 15]
- **Strip the Hypervisor Extension ('H'):** Virtualization logic is unneeded for tightly-coupled edge chips. [16]
- **Target Profile:** Reconfigure the compiler and RTL to target **RV64IMAC** (Integer, Multiply, Atomic, and Compressed instructions). [11]
-

Phase 2: What to Add (The Berkeley Flavor)

UC Berkeley relies heavily on **Chipyard** and the **Gemmini** project to build custom tapeouts. To achieve a similar architecture with CVA6, implement the following: [4, 17, 18]

-
- **Integrate a Quantized Systolic Array:** Use the saved silicon area to add an INT8 or INT4 matrix multiplication accelerator. This mimics Berkeley's Gemmini accelerator.
- **Utilize the Open Source CV-X-IF Interface:** Do not connect the accelerator over a slow peripheral bus (like AXI/AHB). Instead, connect it directly to the CVA6 pipeline using the **Core-V eXtension Interface (CV-X-IF)**. This allows the custom matrix instructions to be fetched, decoded, and dispatched natively by the slimmed CVA6 core.
- **Tightly-Coupled Scratchpad Memories (TCM):** Instead of relying on a large, power-hungry L2 cache, add a dedicated, deterministic instruction and data scratchpad. This guarantees deterministic latency for real-time AI inference pipelines. [4, 9, 14, 17, 19]
-

Silicon Validation & Tapeout Strategy

To successfully realize this project as a test chip, the university team should implement a strict validation matrix:

Stage	Infrastructure Tools	Success Metric
1. Simulation	Verilator / Synopsys VCS	Successfully run CoreMark using only the RV64IMAC toolchain.
2. Co-Design	Spike RISC-V ISA Simulator	Confirm custom Matrix instruction decoding matches expected ISA software behavior.
3. Prototyping	AMD/Xilinx Kintex UltraScale+ FPGA	Synthesize the core + accelerator; run deterministic matrix workloads.
4. ASIC Flow	Open-source Hammer flow or Commercial EDA	Synthesize targeting a multi-project wafer (MPW) run (e.g., SkyWater 130nm or

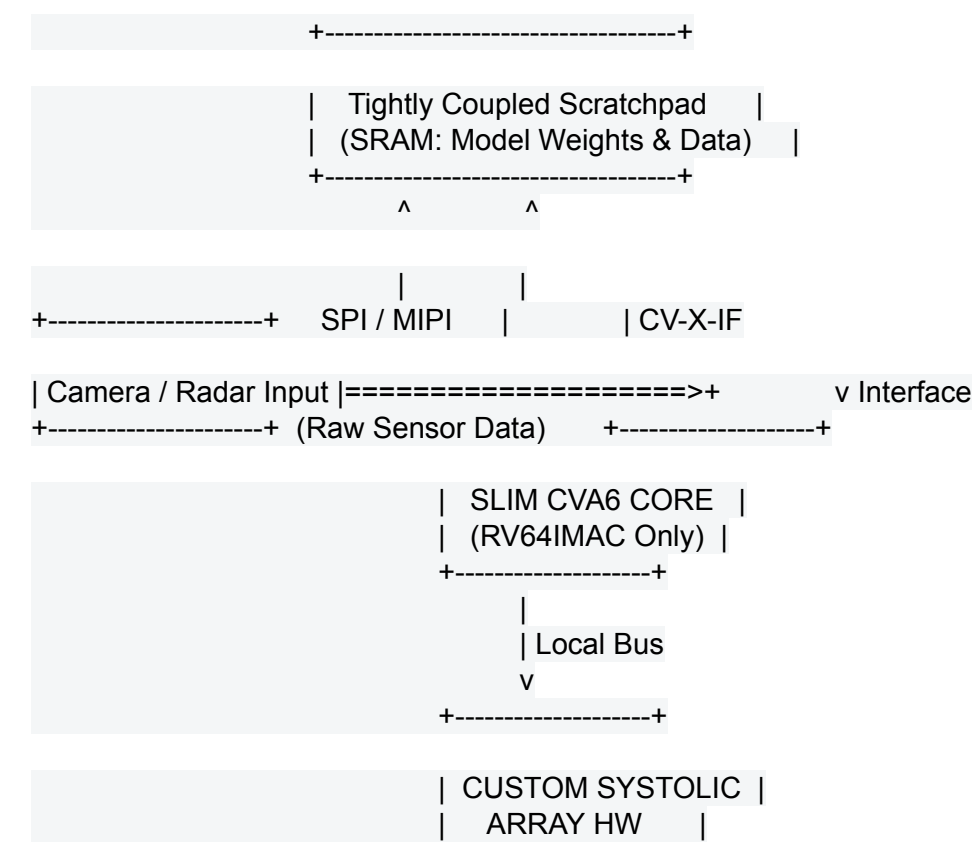
Stage	Infrastructure Tools	Success Metric
		TSMC 65nm).

Expected Project Outcomes

1. **PPA Optimization:** By dropping the FPU, MMU, and Virtualization, the core's silicon area will decrease dramatically (~30-40% reduction of the base CVA6 envelope), easily freeing up the area required for a \$16 \times 16\$ INT8 Systolic Array.
2. **Academic Impact:** This directly replicates the architectural philosophies taught in advanced Berkeley tracks: building lean control planes that drive heavily optimized, specialized execution pipelines. [2, 20]

To make the **Slim-CVA6 Edge-AI SoC** stand out to the university and peer institutions like UC Berkeley, the chip should target an application requiring **high-throughput, ultra-low-latency matrix math** while operating within a strict power budget. [1]

An ideal application for this tapeout is **Always-On Autonomous Drone Obstacle Avoidance via Quantized CNNs**.



| (INT8 / INT4 Math) |
+-----+

Why This Application Fits the "Slimmed" Silicon Profile

- **Drones Require Determinism, Not Virtualization:** Micro-UAVs cannot tolerate the unpredictable execution latencies introduced by OS page faulting, MMU context switching, or massive multi-level cache misses. Removing the MMU and FPU from the base CVA6 ensures completely deterministic real-time loop execution. [2, 3]
- **Heavy Matrix Dominance:** Deep Learning algorithms used for spatial depth sensing, localization, and immediate object bounding boxes are mostly repetitive Multiply-Accumulate (MAC) routines. The slim core handles navigation logic, while the custom systolic array handles the vision network. [4, 5]
- **Severe Resource Constraints:** A custom silicon test chip must remain compact to keep tapeout costs low. Dropping the unnecessary instructions allows the chip to run on standard embedded micro-batteries.

Implementation Details: The Neural Network Pipeline

Instead of a generic processor running heavy software, the software and hardware are completely co-designed: [6]

1. Target Workload

Deploy an optimized version of **MobileNetV2-SSD** or a custom **Tiny-YOLOv8** model, fully quantized to **INT8 weights and INT4 activations**. [4, 7]

2. The Custom ISA Extension (CV-X-IF)

The university team can write custom assembly instructions that interface natively over the CV-X-IF coprocessor bus: [8]

-
- **MAT_LOAD [reg_addr]:** Directly loads the camera's raw pixel values from the Data Scratchpad into the Accelerator's weight-stationary buffers.
- **MAT_MUL [reg_A, reg_B]:** Triggers the 16×16 systolic array to compute a matrix dot product in hardware over a single clock cycle, bypassing standard CPU registers.
- **MAT_ACT_RELU:** Leverages hardware-level activation circuits integrated natively at the edge of the array. [7]
-

3. Execution Pipeline Workflow

3. **Inbound Stream:** Sensor data from a mini-camera or mmWave radar streams directly

- into the internal Scratchpad Memory over a lightweight peripheral interface. [7]
4. **Slim CVA6 Control:** The core encounters the custom MAT_LOAD instruction, staging the neural network layers. [7]
 5. **Hardware Acceleration:** The matrix instructions activate the systolic array, processing video frames to predict immediate obstacles. [5]
 6. **Immediate Flight Correction:** Because the core bypasses heavy floating-point calculation and OS layers, flight adjustment commands (PWM signals sent to drone motors) happen in **sub-millisecond intervals**.

Expected Tapeout Validation Setup

-
- **On-Chip Target:** The core, custom systolic array, and 128 KB of static SRAM blocks.
- **External Integration:** Pin-muxed out to basic SPI, I2C, and GPIO.
- **The Demo Bench:** To mimic UC Berkeley's physical validation style, the university team can mount their fabricated silicon test chip onto a custom PCB daughterboard, connect it to a standard commercial drone frame, and benchmark real-time hardware inference frames-per-second (FPS) versus milliwatts consumed. [9]
-